Finançat per la Unió Europea

IES MESTRE RAMON ESTEVE

GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura, Universitats i Ocupació

UP4.4. CONTROL DE VERSIONES CON GIT

DESPLIEGUE DE APLICACIONES WEB

2º CFGS – DESARROLLO DE APLICACIONES WEB

RA6. Elabora la documentación de la aplicación web evaluando y seleccionando herramientas de generación de documentación, control de versiones y de integración continua.

PABLO PÉREZ GONZÁLEZ

CURSO 2025/26

ÍNDICE

1. **CONTROL DE VERSIONES.**
   - INTRODUCCIÓN.
   - DEFINICIÓN.
2. **REPOSITORIO.**
   - CONCEPTOS CLAVE.
   - CLASIFICACIÓN.
3. **GIT.**
   - INTRODUCCIÓN.
   - ESTADOS.
   - SECCIONES.
   - FLUJO DE TRABAJO BÁSICO.
   - TIPOS DE FLUJO DE TRABAJO.
   - INSTALACIÓN Y CONFIGURACIÓN.
   - COMANDOS BÁSICOS.
   - USO DE GIT CON VISUAL STUDIO CODE.

RESULTADO DE APRENDIZAJE 6

CRITERIO DE EVALUACIÓN e) y g)

RA6. Elabora la documentació de l'aplicació web avaluant i seleccionant ferramentes de generació de documentació, control de versions i d'integració contínua.		
e)	S'ha instal·lat, configurat i utilitzat un sistema de control de versions.	15%
g)	S'ha documentat la instal·lació, configuració i ús del sistema de control de versions utilitzat.	10%

CONTROL DE VERSIONES

INTRODUCCIÓN

En los **proyectos de software**, el código fuente es la joya de la corona, un activo cuyo valor debe protegerse. Para la mayoría de los equipos de software, el código fuente es un repositorio de conocimiento y comprensión invaluables sobre el dominio del problema, que los desarrolladores han recopilado y perfeccionado mediante un minucioso esfuerzo.

El **control de versiones** protege el código fuente tanto de catástrofes como de deterioros accidentales causados por errores humanos y consecuencias accidentales.

CONTROL DE VERSIONES

DEFINICIÓN

Necesidad: Un sistema de control de versiones es esencial al implementar cualquier aplicación mínimamente compleja, ya que ofrece a los desarrolladores la opción de deshacer cambios y volver fácilmente a versiones anteriores del desarrollo.

Propósito: El control de versiones permite mantener un registro de los cambios en un conjunto de documentos a lo largo del tiempo, donde cada versión del documento se identifica con un número, la fecha y hora de generación de la revisión y el nombre de la persona que realizó los cambios.

Repositorio: Es el lugar donde se guardan los conjuntos de cambios. Suele ser un directorio donde se encuentran todos los archivos del proyecto con su número de versión. Cada vez que se añaden cambios al repositorio, se crea una nueva entrada en el historial de cambios. Es necesario distinguir entre repositorios locales, centralizados y/o distribuidos, que se sincronizarán para que diferentes usuarios puedan trabajar en el mismo proyecto y los mismos archivos.

REPOSITORIO

CONCEPTOS CLAVE

1. **Repositorio:** Lugar de almacenamiento para las diferentes versiones del software que el equipo de trabajo está desarrollando.
2. **Rama (branch):** Cada programador/a tiene su copia de trabajo local y realiza modificaciones en una rama específica del proyecto. Si los cambios son correctos, se pueden subir al repositorio notificando el cambio.
3. **Tronco (trunk):** La única línea de desarrollo que no es una rama (a veces también llamada *base line*, *master* o *main*).
4. **Encabezado (head):** Se refiere a la última confirmación, ya sea en el tronco o en una rama.
5. **Copia de trabajo:** Copia local de los archivos del repositorio.
6. **Conflicto:** Ocurre cuando varios programadores/as han realizado varios cambios en la misma rama del software y estos son contradictorios.
7. **Cambio:** Se produce cuando la parte asignada a un programador/a se ha modificado y se sube al repositorio, controlando este cambio mediante el control de versiones.

REPOSITORIO

CONCEPTOS CLAVE

8. **Revisión:** Es una versión del software. Permite el control de versiones para controlar las diferentes revisiones realizadas.
9. **Confirmación (commit):** Confirmación de que se han realizado varios cambios en el mismo documento.
10. **Etiqueta (tag):** Alias utilizado para referirse a una confirmación sin conocer su hash. Se suele usar para indicar números de versión o nombres asociados a una confirmación.
11. **Verificación (checkout):** Crea una copia de trabajo local del repositorio. Un usuario puede especificar una revisión específica u obtener la última.
12. **Bifurcación (fork):** Consiste en crear un nuevo repositorio a partir de otro. Este repositorio no está vinculado al repositorio original y se trata como un repositorio diferente.
13. **Clonación (clone):** Crea un nuevo repositorio que es una copia idéntica de otro.
14. **Extracción (pull):** Es la acción de copiar cambios de un repositorio al repositorio local (esta acción puede causar conflictos).
15. **Inserción (push) o recuperación (fetch):** Acciones utilizadas para añadir cambios del repositorio local a otro repositorio (esta acción puede causar conflictos).

REPOSITORIO

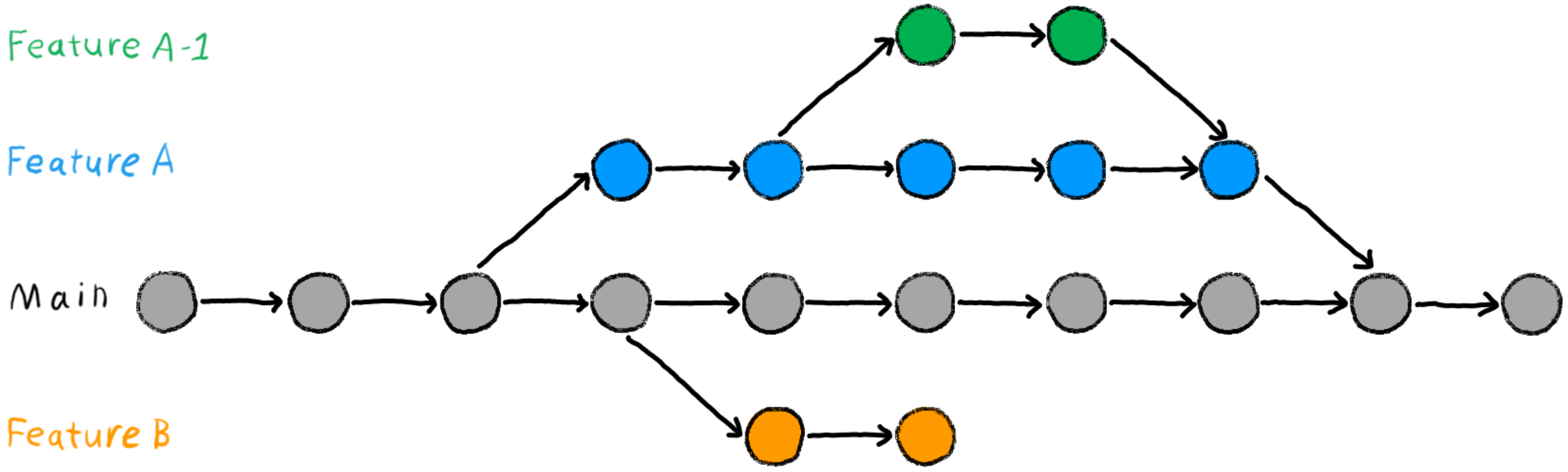
CONCEPTOS CLAVE

16. **Actualización o sincronización (sync):** Acción de combinar los cambios realizados en el repositorio con la copia de trabajo local.
17. **Bloqueo (lock):** Algunos sistemas de control de versiones, en lugar de utilizar el sistema de fusión, bloquean los archivos en uso, así solo un usuario pueda modificarlos a la vez.
18. **Versión:** Conjunto de cambios en un momento específico. Se crea una versión cada vez que se añaden cambios a un repositorio.
19. **Fusión (merge):** La fusión es una operación en la que se aplican dos tipos de cambios a un archivo o conjunto de archivos. Algunos ejemplos son los siguientes:
    - Un usuario que trabaja en un conjunto de archivos actualiza o sincroniza su copia de trabajo con los cambios realizados y confirmados por otros usuarios en el repositorio.
    - Un usuario intenta confirmar archivos que otros usuarios han actualizado desde la última implementación, y el software de control de versiones confirma automáticamente los archivos (tras preguntar al usuario si desea continuar con la integración automática).
    - Se bifurca un conjunto de archivos, se trabaja en un problema existente antes de la rama en una nueva rama y la solución se fusiona en la otra rama.
    - Se crea una rama, se edita el código y posteriormente se fusiona en un único tronco.

REPOSITORIO

CONCEPTOS CLAVE

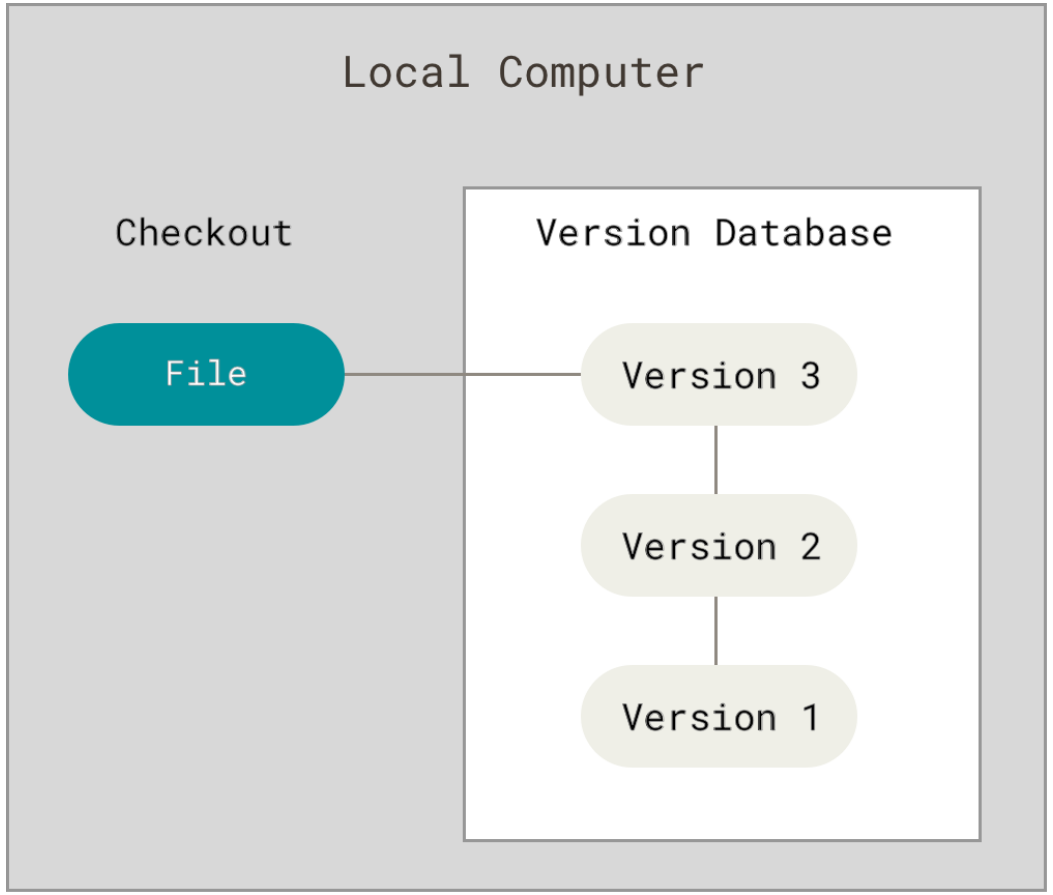
Representación de **revisiones de repositorios con múltiples ramas**:



REPOSITORIO

CLASIFICACIÓN

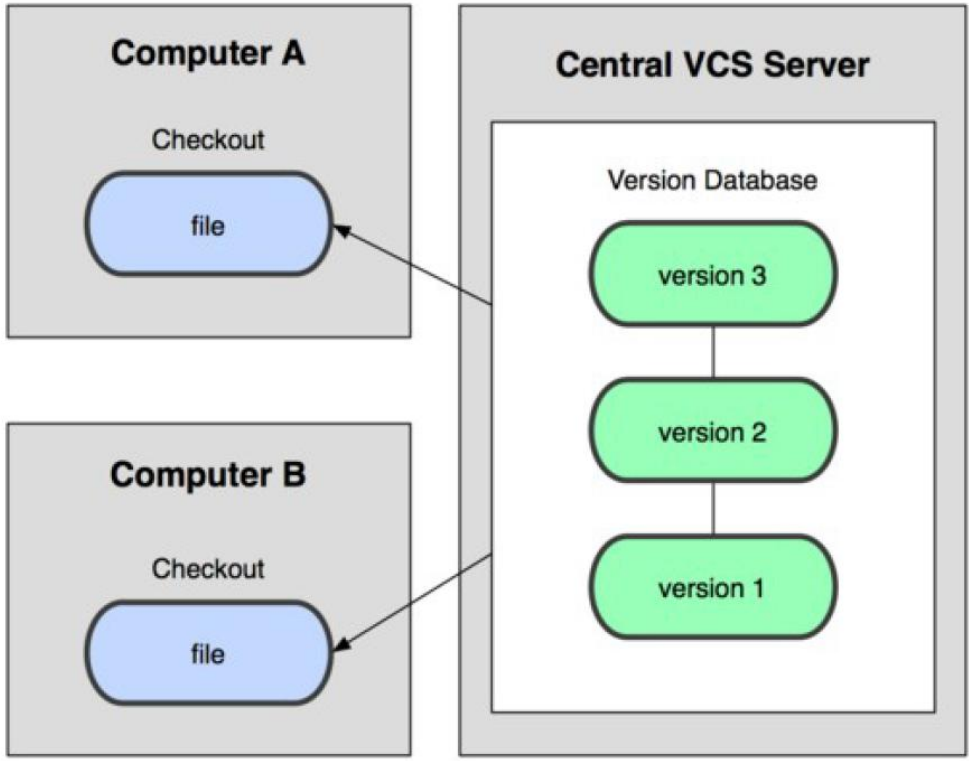
- **Local**: Los cambios se guardan localmente y no se comparten. Esta arquitectura es la precursora de los repositorios remotos y distribuidos.



REPOSITORIO

CLASIFICACIÓN

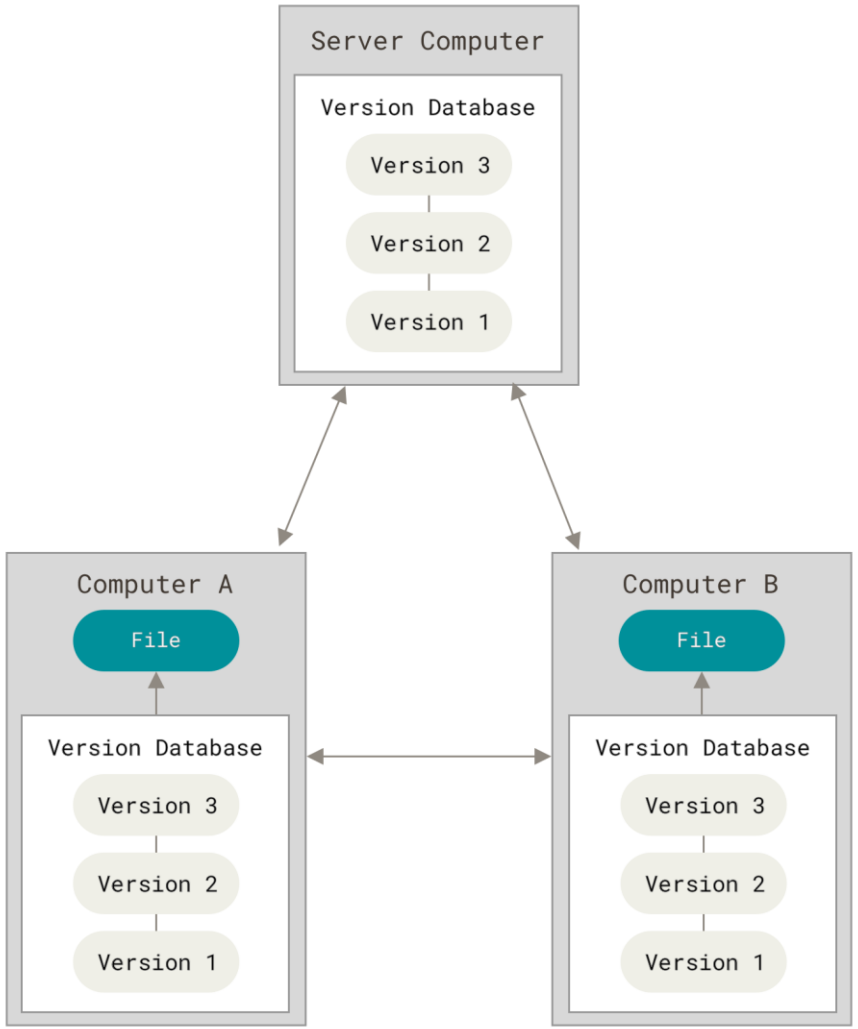
- **Centralizado**: Existe un repositorio centralizado de todo el código, con una sola persona a cargo. Las tareas administrativas se simplifican a cambio de una menor flexibilidad, ya que todas las decisiones importantes (como la creación de una nueva rama) requieren la aprobación del administrador del repositorio.



REPOSITORIO

CLASIFICACIÓN

- **Distribuido:** Cada usuario tiene su propio repositorio. Diferentes repositorios pueden intercambiar y fusionar revisiones. Es común usar un repositorio central, que sirve como punto de sincronización para los diferentes repositorios locales.
- **Ventajas**:
  1. No es necesario iniciar sesión para guardar los cambios.
  2. Posibilidad de seguir trabajando si no se puede acceder al repositorio remoto.
  3. El repositorio central no tiene ramas de prueba.
  4. Se requieren menos recursos para el repositorio remoto.
  5. Mayor flexibilidad al permitirle gestionar cada repositorio personal como desee.



GIT

INTRODUCCIÓN

- Creado por Linus Torvalds, **Git** es el repositorio de código más popular entre los programadores de todo el mundo. Es la primera opción de casi el 90% de los desarrolladores, seguido a gran distancia por Apache Subversion.
- **Git** se diferencia del resto en la forma en que modela sus datos. La mayoría de los demás sistemas almacenan la información como una lista de cambios basados en archivos, mientras que Git modela sus datos como un conjunto de instantáneas de un mini-sistema de archivos.

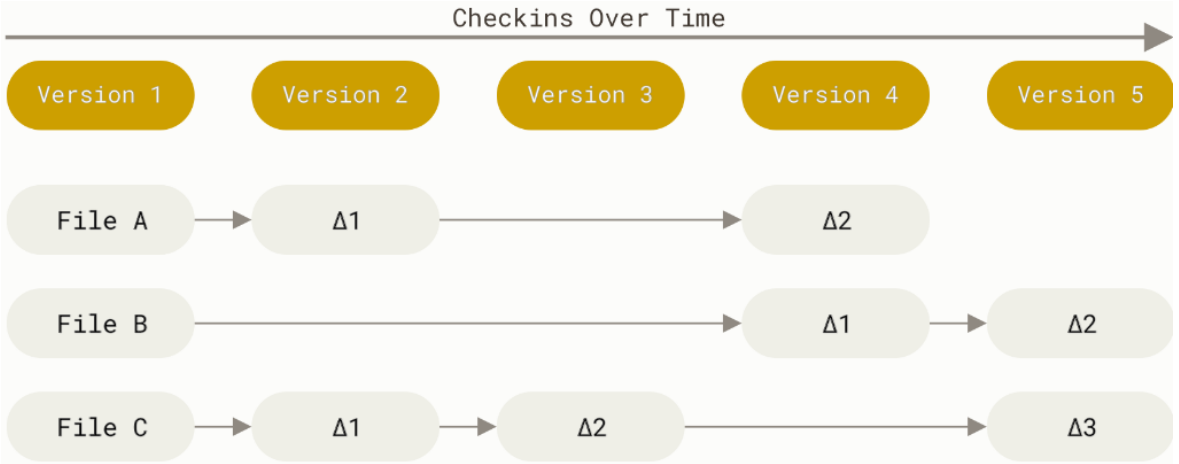


Figure 4. Storing data as changes to a base version of each file

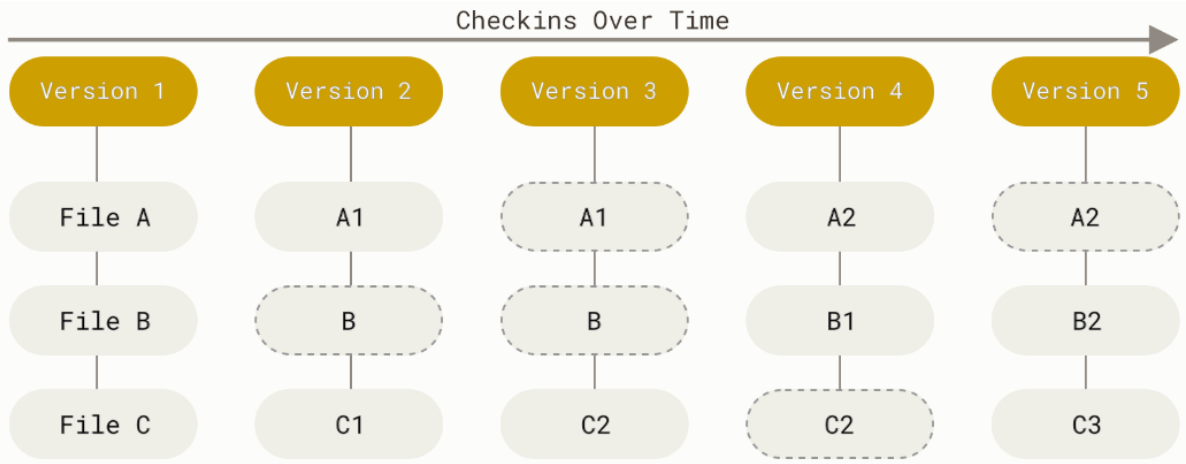


Figure 5. Storing data as snapshots of the project over time

GIT

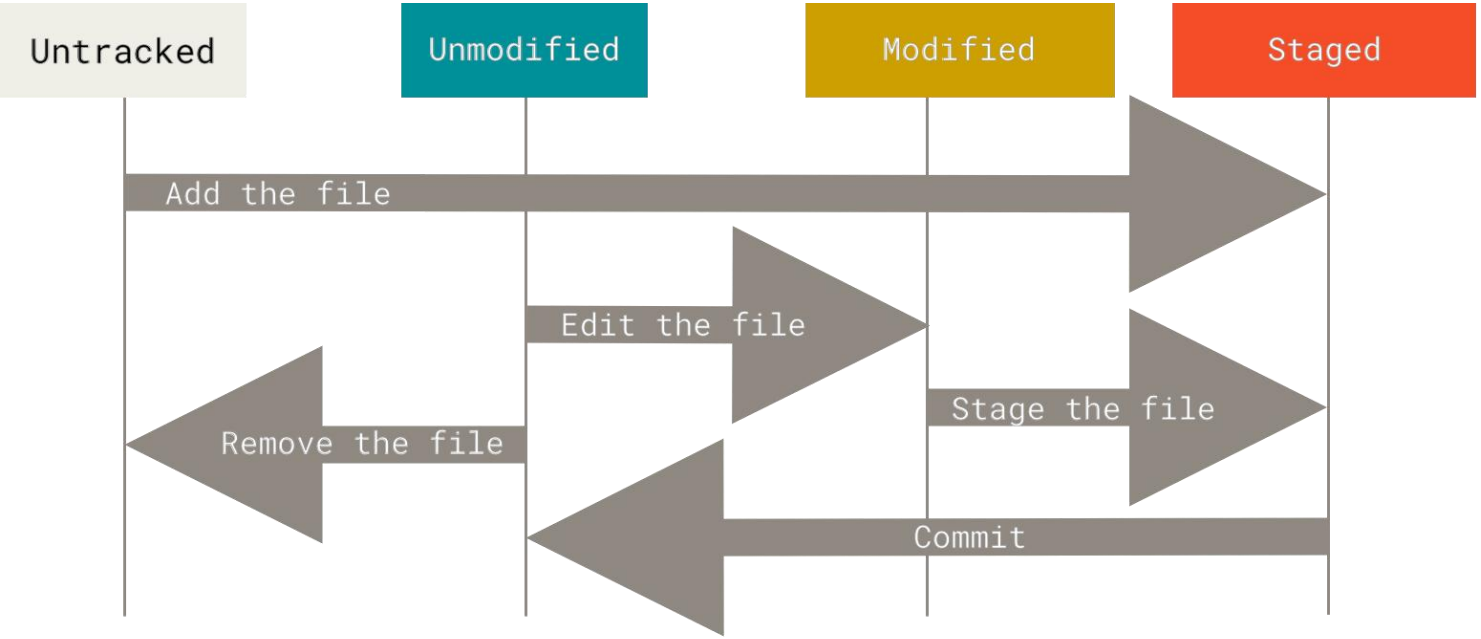
INTRODUCCIÓN

- Entre las **ventajas** de usar Git, se encuentra la posibilidad de trabajar con el repositorio de código abierto más grande del mundo, **GitHub**, que además permite compartir código de forma fácil y sencilla.
- Algunas de las **ventajas** de GitHub son:
  - Seguimiento de errores.
  - Búsqueda rápida.
  - Potente comunidad de desarrolladores.
  - Permite descargar el código fuente como archivo.
  - Permite importar a Git, SVN o TFS.
  - Personaliza cualquier servicio de alojamiento en la nube.

<u>Nota</u>: SVN y TFS son Subversion y Team Foundation Server (sistemas de control de versiones).

GIT
ESTADOS

- Git tiene **tres estados principales**: modificado, preparado y confirmado:
- **Modificado** significa que has modificado el archivo, pero aún no lo has confirmado en tu base de datos. **Preparado** significa que has marcado un archivo modificado en su versión actual para incluirlo en tu próxima instantánea de confirmación. **Confirmado** significa que los datos están almacenados de forma segura en tu base de datos local.



GIT

SECCIONES

Este estado nos lleva a las tres **secciones principales de un proyecto Git**:

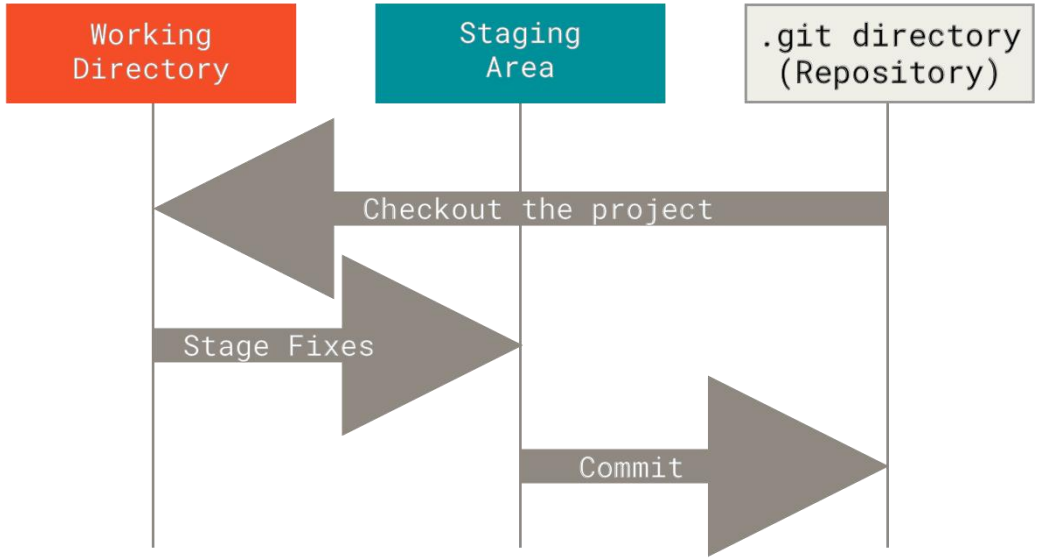
- El **árbol de trabajo** es una única versión del proyecto. Estos archivos se extraen de la base de datos comprimida en el directorio de Git y se almacenan en el disco para su uso o modificación.
- El **área de preparación** es un archivo, generalmente ubicado en el directorio de Git, que almacena información sobre lo que se incluirá en la próxima confirmación. Su nombre técnico en Git es "*index*", pero la frase "*staging area*" también se utiliza.
- El **directorio de Git** es donde Git almacena los metadatos y la base de datos de objetos del proyecto. Esta es la parte más importante de Git y es lo que se copia al clonar un repositorio desde otro equipo.

GIT

FLUJO DE TRABAJO BÁSICO

El **flujo de trabajo básico de Git** es:

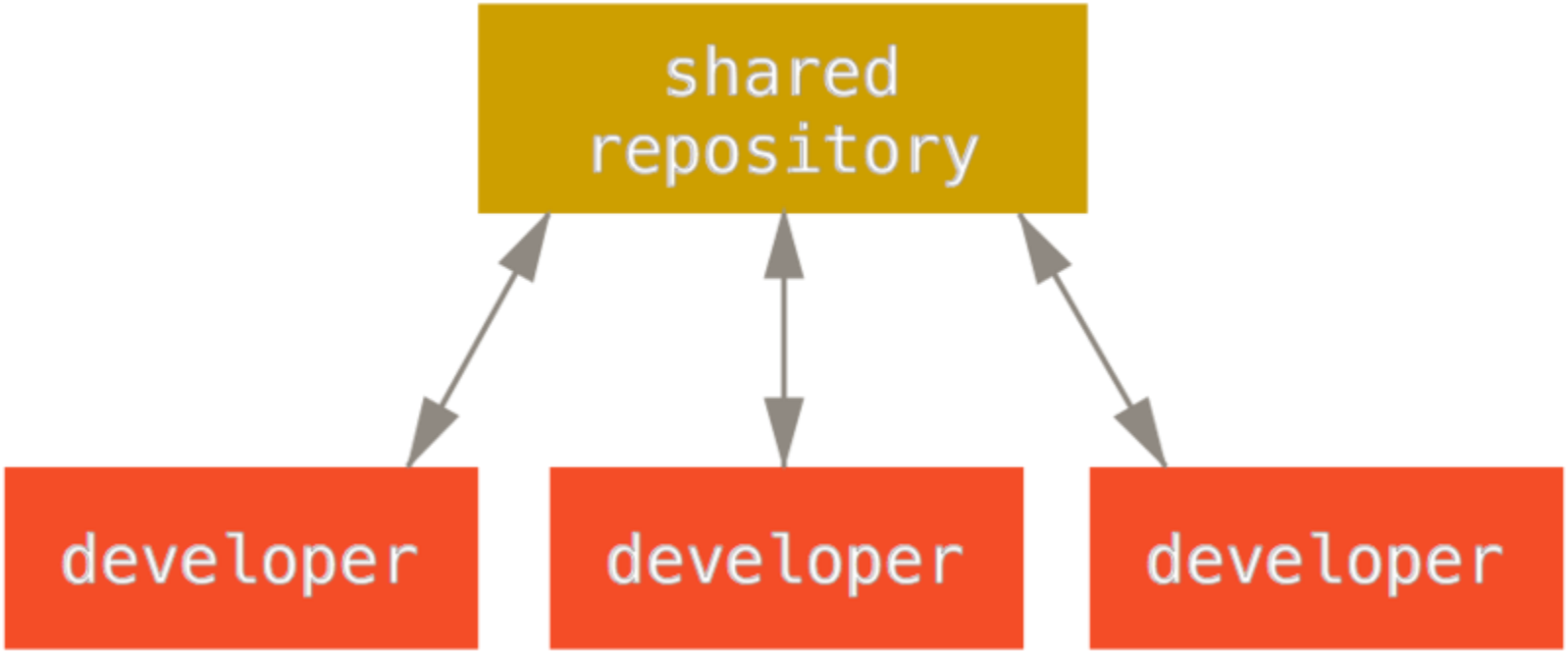
- Modificas archivos en tu **árbol de trabajo**.
- Preparas selectivamente solo los cambios que quieres que formen parte de tu próxima confirmación, lo que añade solo esos cambios al **área de preparación**.
- Realizas una confirmación, que toma los archivos tal como están en el área de preparación y almacena esa instantánea permanentemente en tu **directorio de Git**.



GIT

TIPOS DE FLUJO DE TRABAJO

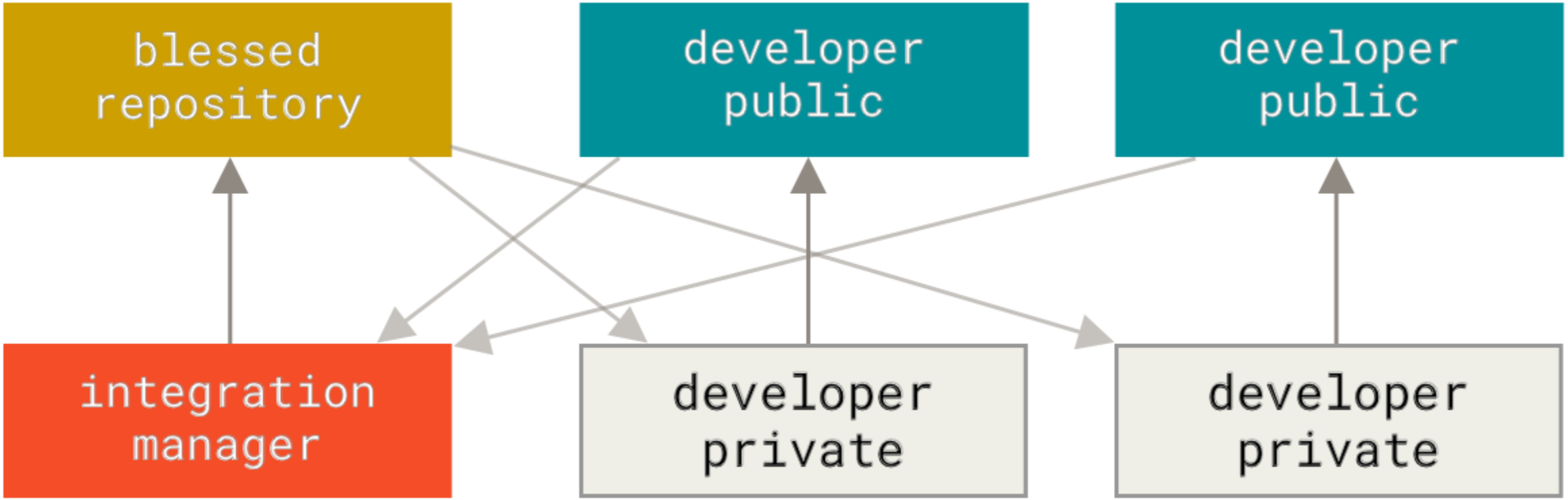
- **Flujo de trabajo centralizado:** Solo hay un repositorio que almacena el código y todos sincronizan su trabajo con él. Esto significa que, si dos desarrolladores clonan desde el centro y ambos realizan cambios, el primero que los publique puede hacerlo sin problemas. El segundo desarrollador debe integrar el trabajo del primero antes de publicar los cambios para no sobrescribirlos.



GIT

TIPOS DE FLUJO DE TRABAJO

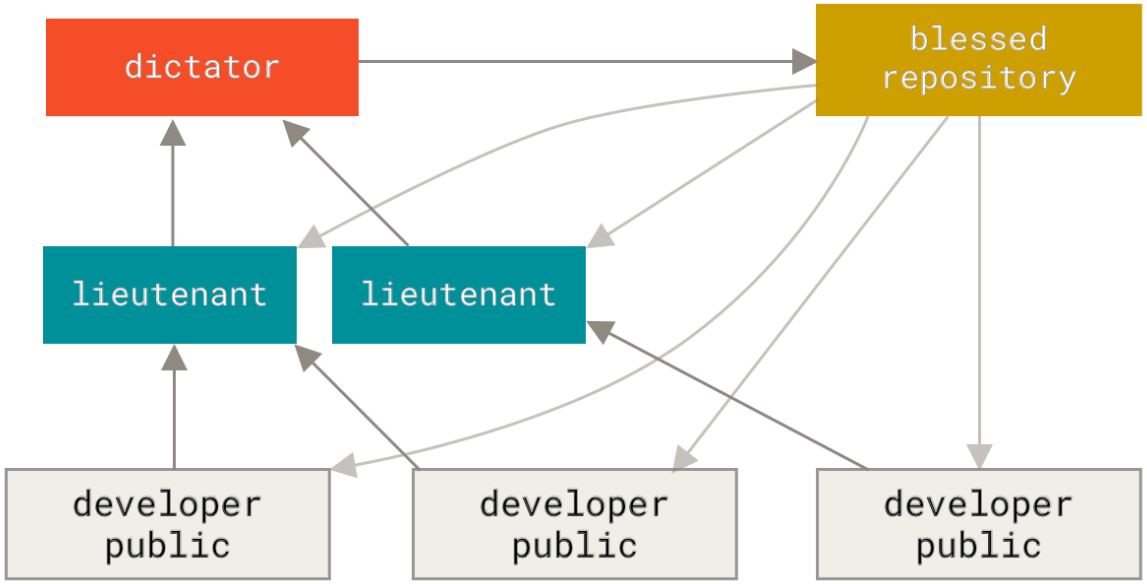
- **Flujo de trabajo Integration-Manager:** Al habilitar múltiples repositorios remotos, Git permite un flujo de trabajo donde cada desarrollador tiene acceso de escritura a su propio repositorio público y acceso de lectura a los repositorios de los demás. Este escenario suele incluir un repositorio canónico que representa el proyecto "oficial".



GIT

TIPOS DE FLUJO DE TRABAJO

- **Flujo de trabajo Dictador-Tenientes:** Es una variante del flujo de trabajo de múltiples repositorios. Generalmente se utiliza en proyectos grandes con cientos de colaboradores; un ejemplo conocido es el kernel de Linux. Varios gestores de integración supervisan ciertas partes del repositorio; se les llama *tenientes*. Todos los tenientes tienen un gestor de integración conocido como el *dictador*. Este dictador envía datos desde su directorio a un repositorio de referencia del que todos los colaboradores deben extraerlos.



GIT

INSTALACIÓN Y CONFIGURACIÓN

- Página web para instalar Git. Depende de tu Sistema operativo.
- Para comprobar que Git está instalado, podemos ejecutar “**`git -v`**” or “**`git`**”.
- Para configurar Git, debemos ejecutar estos comandos:

```
git config --global user.name “nombre y apellido"
git config --global user.email dirección@de_correo
```

- Si la configuración es correcta, al ejecutar los siguientes comandos, devolverá tus datos.

```
git config --global user.name
```

Tu nombre

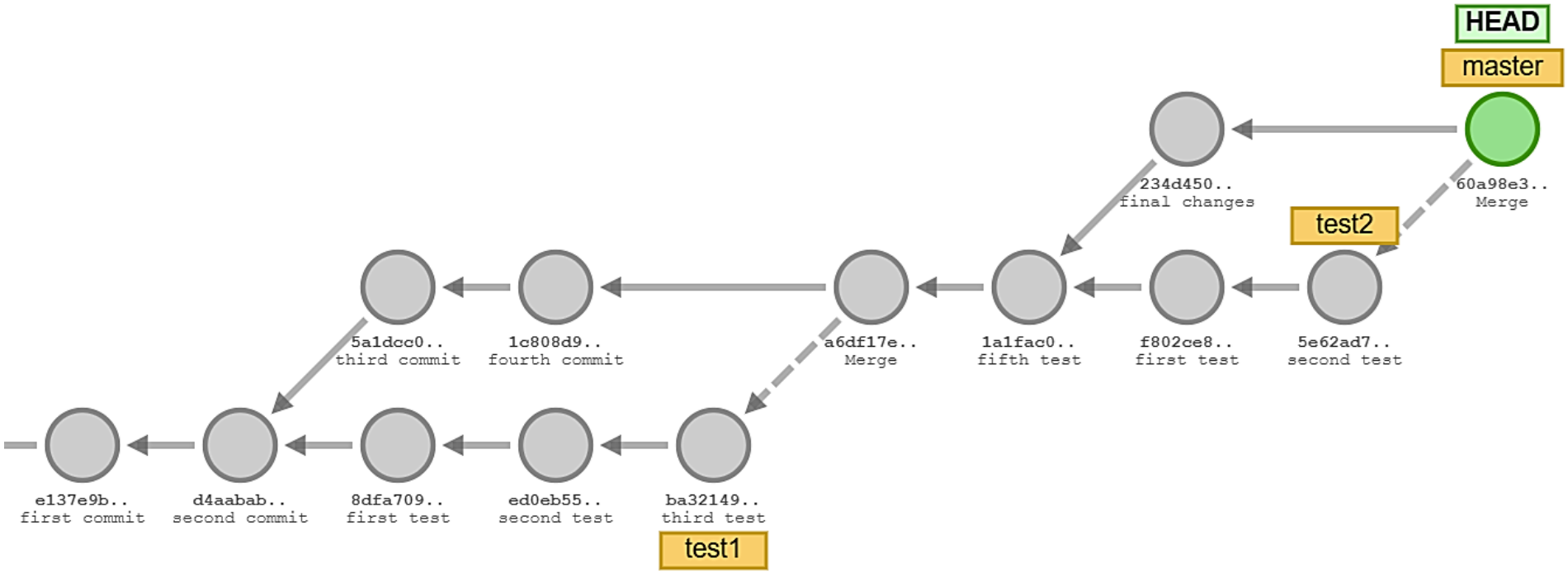
```
git config --global user.email
```

Tu dirección de correo

GIT

COMANDOS BÁSICOS

- Esta reference sheet resume las instrucciones de Git por terminal más utilizadas.
- Accede a la herramienta "Visualizing Git" y realiza la siguiente estructura usando CLI:



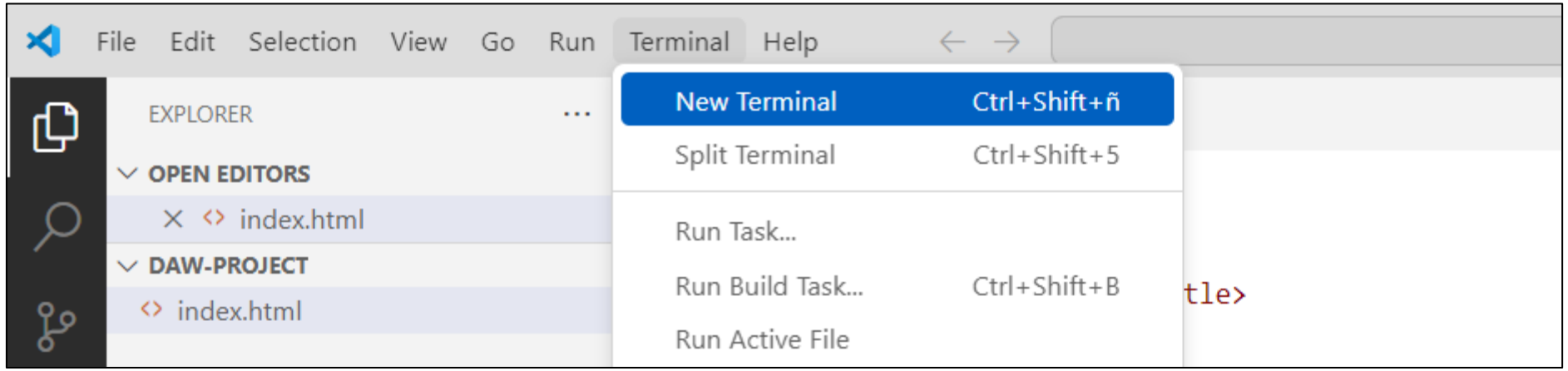
- Además, debes enumerar el historial de versiones maestras, enumerar todas las ramas en el repositorio y eliminar las ramas test1 y test2.

GIT

USO DE GIT CON VISUAL STUDIO CODE

Para **inicializar Git**, debes seguir estos pasos:

1. Crea una carpeta "**`daw-project`**" con tus archivos HTML/PHP.
2. Abre Visual Studio Code y la carpeta "**`daw-project`**".
3. Ara inicializar Git, ves a *Terminal* ➔ *New Terminal*.



4. Visual Studio Code abre una nueva terminal en la parte inferior del editor:

GIT

USO DE GIT CON VISUAL STUDIO CODE

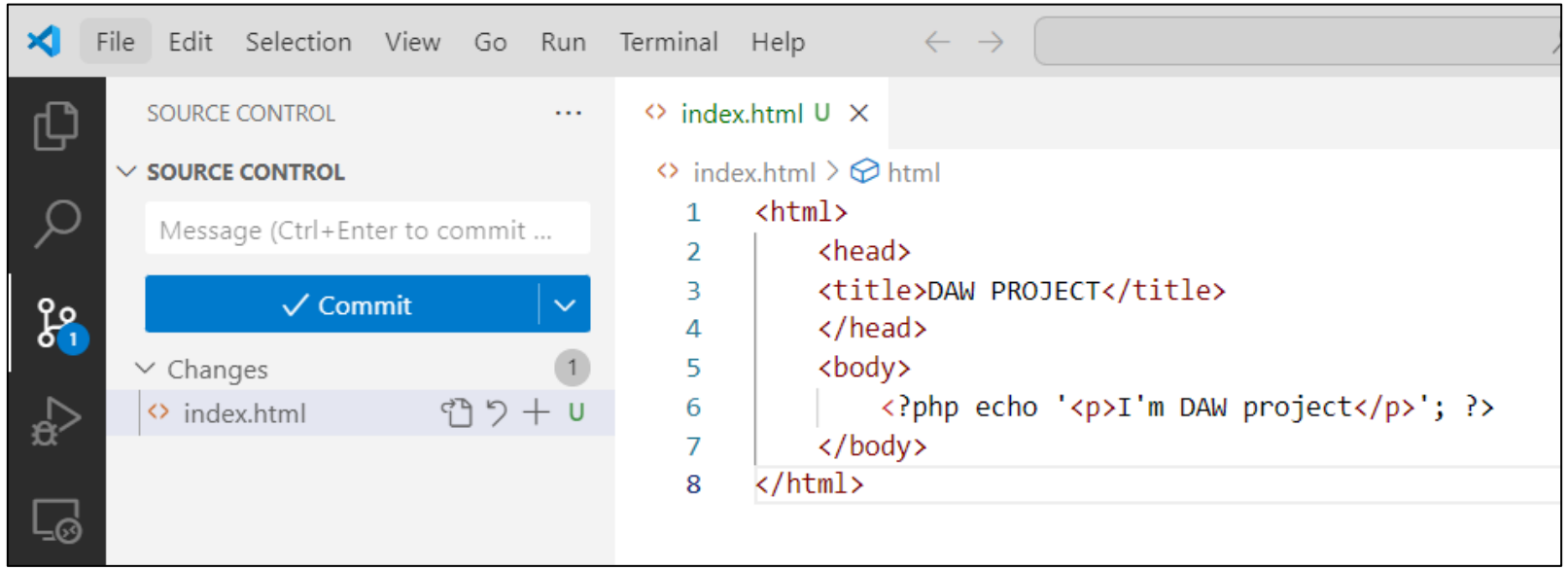
Para **inicializar Git**, debes seguir estos pasos:

5. En la terminal, ejecuta el comando “**`git init`**”:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\pablo\Desktop\daw-project> git init
Initialized empty Git repository in C:/Users/pablo/Desktop/daw-project/.git/
```

6. La pestaña Source Control mostrará la información siguiente:

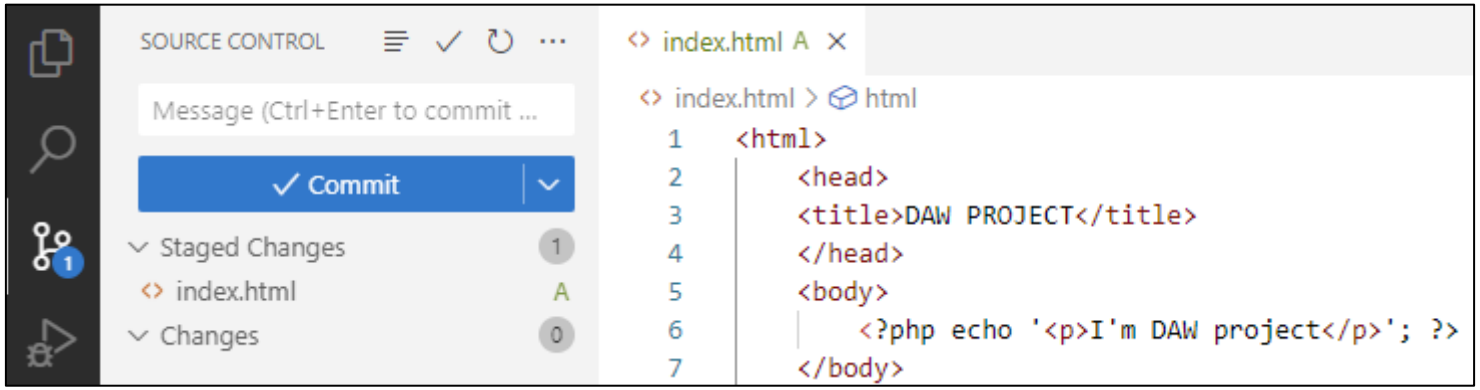


GIT

USO DE GIT CON VISUAL STUDIO CODE

Para realizar un **commit inicial en Git**, debes seguir estos pasos:

1. En la pestaña Source Control, el archivo `index.html` muestra “U” (Untracked). Para preparar los cambios, ves a *Source Control options* ➜ *Changes* ➜ *Stage All Changes*. Entonces, el archivo `index.html` mostrará “A” (Index Added).



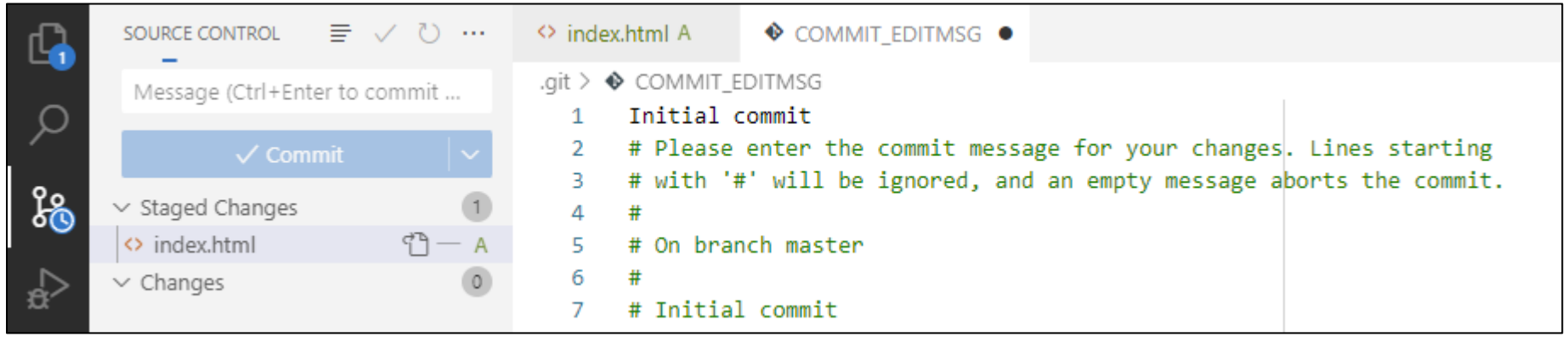
2. Para llevar a cabo el commit inicial, ves a *Source Control options* ➜ *Commit* ➜ *Commit Staged*.

GIT

USO DE GIT CON VISUAL STUDIO CODE

Para realizar un **commit inicial en Git**, debes seguir estos pasos:

3. Debes incluir un mensaje de commit para tus cambios. En este caso, se puede escribir "`Initial commit`" y aceptar los cambios.



4. Todos tus cambios se guardarán, y no tendrás cambios pendientes.

GIT

USO DE GIT CON VISUAL STUDIO CODE

Para realizar un **cambio en Git**, debes seguir estos pasos:

1. Al introducir un cambio en el código `index.html`, el estado del archivo en la pestaña *Source Control* cambia a “M” (Modified). Haciendo click en el archivo `index.html`, mostrará los cambios en dos pestañas diferentes. La pestaña roja contiene el código original y la verde contiene los cambios.

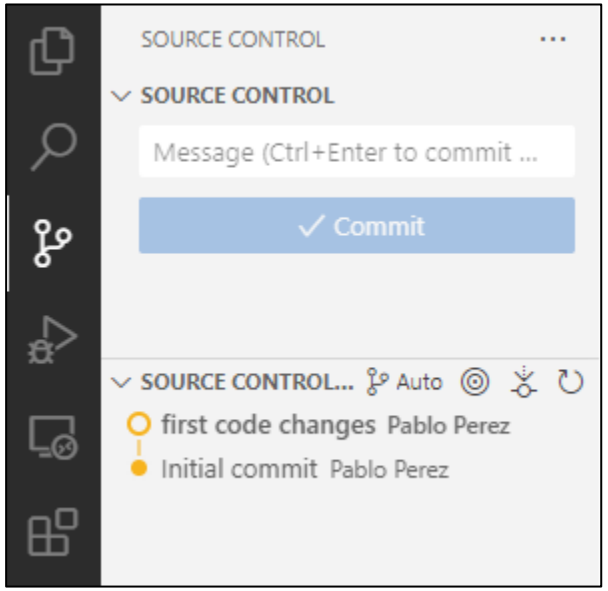
2. Para guardar estos cambios, ves a Source *Control options* ➔ *Changes* ➔ *Stage All Changes*. Entonces, ves a *Source Control options* ➔ *Commit* ➔ *Commit Staged* e incluye un mensaje para el commit con los cambios realizados.

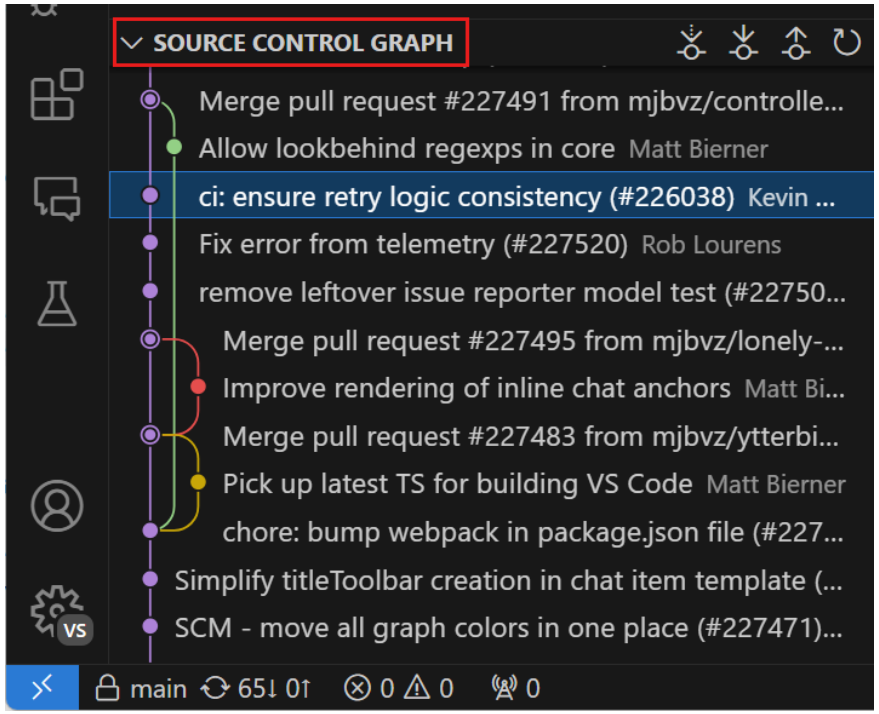
GIT

USO DE GIT CON VISUAL STUDIO CODE

Para ver las **modificaciones del proyecto**, debes seguir estos pasos:

1. Para habilitar el historial del proyecto: *Source Control options* ➜ *Source Control Graph*.
2. La sección *Source Control Graph* de la vista *Source Control* muestra una representación gráfica de los commit entrantes y salientes.





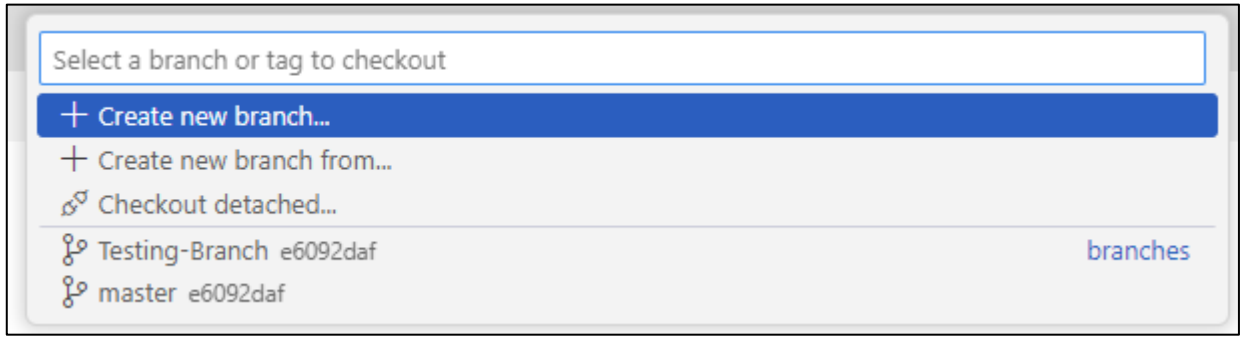
GIT

USO DE GIT CON VISUAL STUDIO CODE

Para **crear y gestionar ramas**, debes seguir estos pasos:

1. Para crear una nueva rama, ves a *Source Control options* ➔ *Branch* ➔ *Create Branch*.
2. Incluye el nombre de la rama. Por ejemplo, "*Testing-Branch*".
3. En la esquina inferior izquierda, aparece la nueva rama "*Testing-Branch*".

4. Si haces click en esta nueva rama, podrás ver todas las ramas que tienes en el proyecto y podrás seleccionar la rama donde quieres trabajar.

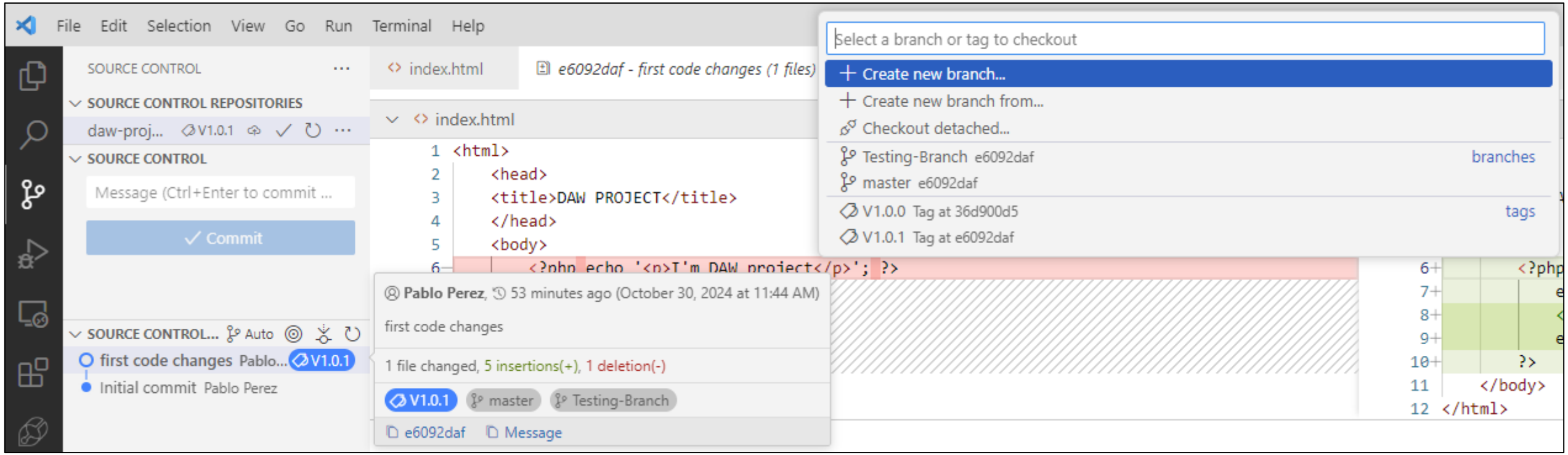


GIT

USO DE GIT CON VISUAL STUDIO CODE

Para **crear y gestionar etiquetas**, debes seguir estos pasos :

1. Para crear una nueva etiqueta, ves a *Source Control options* ➔ *Tags* ➔ *Create Tag*.
2. Incluye los nombres de las etiquetas y un mensaje para anotarlas. Por ejemplo, "*V1.0.0*" para "*Project Creation*" y "*V1.0.1*" para "*First changes in PHP code*".

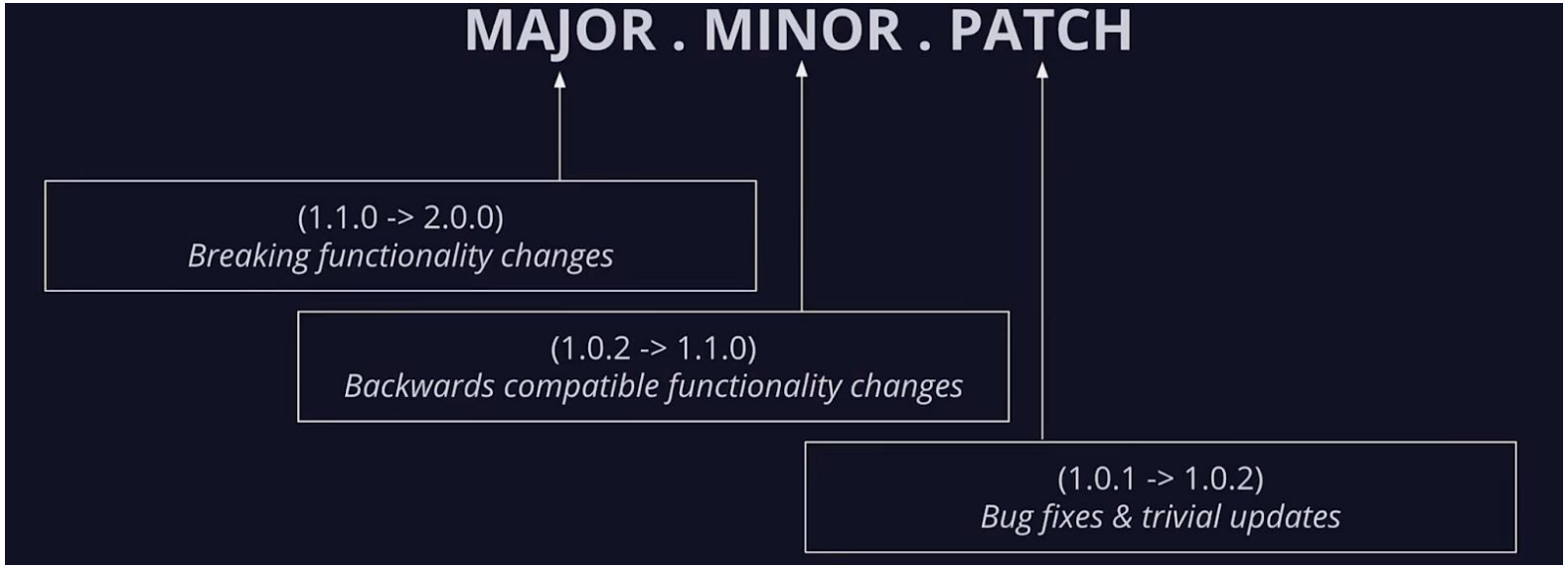


GIT

USO DE GIT CON VISUAL STUDIO CODE

Cuando se utilizan etiquetas, un **número de versión semántico consta de tres partes**:

- **Patch number**: Indica corrección de errores o realización de actualizaciones menores.
- **Minor release**: Indica que se ha producido un cambio en la funcionalidad..
- **Major release**: Indica cambios que podrían afectar la compatibilidad con versiones anteriores o alterar significativamente la funcionalidad.

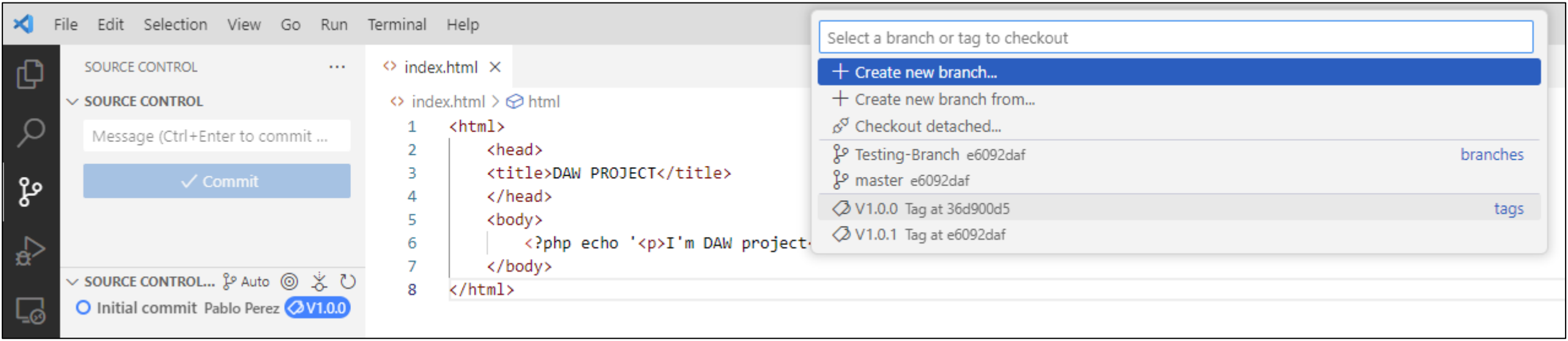


GIT

USO DE GIT CON VISUAL STUDIO CODE

Para **restaurar una versión antigua del proyecto**, debes seguir estos pasos:

1. Para restaurar una versión antigua del proyecto, ves a *Source Control options* ➜ *Checkout to…*
2. Selecciona la versión que deseas restaurar desde la versión de etiqueta.
3. Puedes visualizar el código de la versión seleccionada.

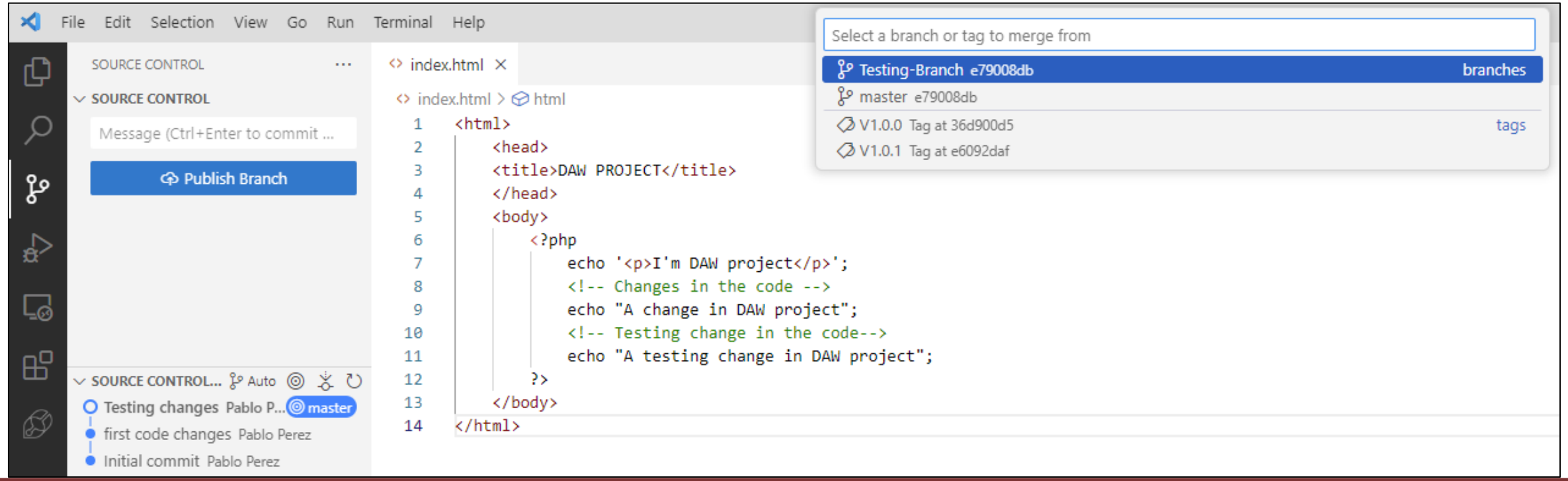


GIT

USO DE GIT CON VISUAL STUDIO CODE

Para **fusionar (merge) 2 ramas** (de *Testing-Branch* a master), debes seguir estos pasos:

1. Ubícate en la rama donde deseas aplicar las modificaciones (`master`).
2. Para fusionar 2 ramas diferentes, ves a *Source Control options* → *Branch* → *Merge…*
3. Seleccione una rama o etiqueta para fusionar (`Testing-Branch`).



GIT

USO DE GIT CON VISUAL STUDIO CODE

Si necesitas más información sobre las diferentes funcionalidades de control de código fuente de Git en VS Code, puedes visitar Using Git source control in VS Code.

PREGUNTAS

DUDAS Y SUGERENCIAS